

InventoryPlus — Architectural Refactoring Plan

Detailed analysis of remaining monolithic elements in Inventory.tsx and extraction roadmap.

Refactoring Status & Stats

Component: Inventory.tsx

- **Original Size:** 1,770 lines (God Component containing all styles, modals, state, icons, and API logic)
- **Current Size:** ~1,280 lines (~28% reduction after extracting Modals & Sidebar)
- **Status:** In Progress / Partial

Remaining Architectural Gaps

Target Part	Current State in Inventory.tsx	Proposed Extraction Destination	Impact & Benefit
1. API / Fetch Logic	Multiple inline async functions (e.g. fetchDepartments, fetchMembers, saveSpecs) handle network requests and state mutations.	src/shared/inventory/hooks/ (Sub-hook breakdown)	Decouples data-fetching logic from UI presentation; allows easier mock testing and cleaner state lifecycle management.
2. Inline SVG Icons	Large chunks of hardcoded SVG elements (like edit icons, checkmarks, delete icons) clog up the top and bottom of the JSX tree.	src/shared/components/icons.tsx	Removes noise from the main UI render tree; standardizes icon sizes/colors and makes icons reusable across other files.
3. Diagnostics / Card Grid UI	Rendering and layout logic for individual workstation cards, diagnostics tables, and status lists are embedded in the main layout.	src/shared/inventory/components/DiagnosticsGrid.tsx	Improves DOM rendering performance; makes it simpler to update spec displays without affecting department layout.

3. Detailed Hooking Breakdown (Sub-Hooks Architecture)

Rather than using one large `useInventoryApi.ts` hook, the state and fetching logic will be broken down into three domain-specific sub-hooks located in `src/shared/inventory/hooks/`:

1. `useDepartments.ts`

- **Responsibility:** Manages the collection of departments loaded into the application.
- **Internal States:** `cards`, `newDeptName`
- **API Methods:** `fetchDepartments()`, `handleAddCardSubmit()`

2. `useMembers.ts` (depends on *`selectedDepartment`*)

- **Responsibility:** Manages the list of members for a selected department and syncs selection history.
- **Internal States:** `members`, `isSubmittingMember`
- **API Methods:** `fetchMembers(deptName)`, `handleAddMemberSubmit(name, email)`
- **Caching:** Restores selection state from `localStorage` username properties.

3. `useWorkstationSpecs.ts` (depends on *`selectedMemberObj`*, *`selectedDepartment`*)

- **Responsibility:** Manages specifications retrieval, card saves, and cache operations.
- **Internal States:** `workstationSpecs`
- **API Methods:** `fetchSpecs()`, `saveSpecsToDb(specs)`
- **State Handlers:** `handleSaveCardEdits(title, items)`, `handleDeleteCard(title)`
- **Caching:** Performs spec sync against the `inventoryplus_workstation_specs` `localStorage` cache.

Composition Orchestrator:

The parent hook `useInventoryApi.ts` will combine all three sub-hooks internally. This composition pattern keeps the parent `Inventory.tsx` component clean, as it only needs to call a single hook to get the entire data context.

4. Step-by-Step Refactoring Roadmap

Step 1: Extract SVG Assets (Low Risk)

- **What to do:** Create a new file: `src/shared/components/icons.tsx`.
- **Action:** Copy all inline SVG functions from the bottom of `Inventory.tsx`. Import and use them in `Inventory.tsx`. This immediately shaves off ~150 lines of static code.

Step 2: Separate State from Style (Medium Risk)

Your UI shouldn't be stressing over network requests. Functions like `fetchDepartments` and `saveWorkstationSpecs` belong in the background.

- **What to do:** Extract all data-fetching and state management into a custom React hook: `src/shared/inventory/hooks/useInventoryApi.ts`.
- **UI/UX Benefit:** By isolating the API logic, the UI component only has to worry about looking good and handling user interactions. It also makes loading states and error skeletons much easier to implement cleanly.

Step 3: Build the Hardware Diagnostics Grid (The Core UI UI/UX)

This is where the actual inventory layout comes to life. You need to isolate the layout logic for individual workstation statuses.

- **What to do:** Create `src/shared/inventory/components/DiagnosticsGrid.tsx`. Move the sections rendering your hardware categories—specifically the Network, CPU, and RAM cards—into this file. Pass the workstation specs down as props.
- **UI/UX Benefit:** Improves DOM rendering performance. If a user updates a CPU spec, only that specific card re-renders instead of jarringly blinking the entire department layout.